Why I Ignore The Spotlight as a Staff Engineer

Dec 4, 2025

Lately I've been reading Sean Goedecke's essays on being a Staff+ engineer. His work (particularly Software engineering under the spotlight and It's Not Your Codebase) is razor-sharp and feels painfully familiar to anyone in Big Tech.

On paper, I fit the mold he describes: I'm a Senior Staff engineer at Google. Yet, reading his work left me with a lingering sense of unease. At first, I dismissed this as cynicism. After reflecting, however, I realized the problem wasn't Sean's writing but my reading.

Sean isn't being bleak; he is accurately describing how to deal with a world where engineers are fungible assets and priorities shift quarterly. But my job looks nothing like that and I know deep down that if I tried to operate in that environment or in the way he described I'd burn out within months.

Instead I've followed an alternate path, one that optimizes for systems over spotlights, and stewardship over fungibility.

We Live in Different Worlds

The foundational reason for our diverging paths is that Sean and I operate in entirely different worlds with different laws governing them.

From Sean's resume, my understanding is that he has primarily worked in product teams [1] building for external customers. Business goals pivot quarterly, and success is measured by revenue or MAU. Optimizing for the "Spotlight" makes complete sense in this environment. Product development at big tech scale is a crowded room: VPs, PMs and UX designers all have strong opinions. To succeed, you *have* to be agile and ensure you are working specifically on what executives are currently looking at.

On the other hand, I've spent my entire career much more behind the scenes: in developer tools and infra teams.

My team's customers are thousands of engineers in Android, Chrome, and throughout Google [2]. End users of Google products don't even know we exist; our focus is on making sure developers have the tools to collect product and performance metrics and debug issues using detailed traces.

In this environment, our relationship with leadership is very different. We're never the "hot project everyone wants," so execs are not fighting to work with us. In fact, my team has historically struggled to hire PMs. The PM career ladder at Google incentivizes splashy external launches so we cannot provide good "promotion material" for them. Also, our feedback comes directly from engineers. Adding a PM in the middle causes a loss in translation, slowing down a tight, high-bandwidth feedback loop.

All of this together means our team operates "bottom-up": instead of execs telling us "you should do X", we figure out what we think will have the most impact to our customers and work on building those features and tools. Execs ensure that we're *actually* solving these problems by considering our impact on more product facing teams.

Compounding Returns of Stewardship

In the product environments Sean describes, where goals pivot quarterly and features are often experimental, speed is the ultimate currency. You need to ship, iterate, and often move on before the market shifts. But in Infrastructure and Developer Experience, context is the currency.

Treating engineers as fungible assets destroys context. You might gain fresh eyes, but you lose the implicit knowledge of how systems actually break. Stewardship, staying with a system long-term, unlocks compounding returns that are impossible to achieve on a short rotation.

The first is efficiency via pattern matching. When you stay in one domain for years, new requests are rarely truly "new." I am not just debugging code; I am debugging the intersection of my tools and hundreds of diverse engineering teams. When a new team comes to me with a "unique" problem, I can often reach back in time: *"We tried this approach in 2021 with the Camera team; here is exactly why it failed, and here is the architecture that actually works".*

But the more powerful return is systemic innovation. If you rotate teams every year, you are limited to solving acute bugs that are visible *right now*. Some problems, however, only reveal their shape over long horizons.

Take Bigtrace, a project I recently led; it was a solution that emerged solely because I stuck around long enough to see the shape of the problem:

- Start of 2023 (Observation): I began noticing a pattern. Teams across Google were collecting terabytes or even petabytes of performance traces, but they were struggling to process them. Engineers were writing brittle, custom pipelines to parse data, often complaining about how slow and painful it was to iterate on their analysis.

- Most of 2023 (Research): I didn't jump to build a production system. Instead, I spent the best part of a year prototyping quietly in the background while working on other projects. I gathered feedback from these same engineers who had complained and because I had established long-term relationships, they gave me honest and introspective feedback. I learned what sort of UX, latency and throughput requirements they had and figured out how I could meet them.

- End of 2023 to Start of 2024 (Execution): We built and launched Bigtrace, a distributed big data query engine for traces. Today, it processes over 2 billion traces a month and is a critical part of the daily workflow for 100+ engineers.

If I had followed the advice to "optimize for fungibility" (i.e. if I had switched teams in 2023 to chase a new project) Bigtrace would not exist.

Instead, I would have left during the research phase and my successor would have seen the same "noise" of engineers complaining. But without the historical context to recognize a missing puzzle piece, I think they would have struggled to build something like Bigtrace.

The Power of "No"

One of the most seductive arguments for chasing the "Spotlight" is that it guarantees resources and executive attention. But that attention is a double-edged sword.

High-visibility projects are often volatile. They come with shifting executive whims, political maneuvering, and often end up in situations where long-term quality is sacrificed for short-term survival. For some engineers, navigating this chaos is a thrill. For those of us who care about system stability, it feels like a trap.

The advantage of stewardship is that it generates a different kind of capital: trust. When you have spent years delivering reliable tools, you earn the political capital to say "No" to the spotlight when it threatens the product.

Recently, the spotlight has been on AI. Every team is under pressure to incorporate it. We have been asked repeatedly: *"Why don't you integrate LLMs into Perfetto?"* If I were optimizing for visibility, the answer would be obvious: build an LLM wrapper, demo it to leadership, and claim we are "AI-first." It would be an easy win for my career.

But as a steward of the system, I know that one of Perfetto's core values is precision. When a kernel developer is debugging a race condition, they need exact timestamps, not a hallucination. Users trust that when we tell them "X is the problem" that it actually *is* the problem and they're not going to go chasing their tail for the next week, debugging an issue which doesn't exist.

But it's important not to take this too far: skepticism shouldn't become obstructionism. With AI, it's not "no forever" but "not until it can be done right" [3].

A spotlight-seeking engineer might view this approach as a missed opportunity; I view it as protecting what makes our product great: user trust.

The Alternate Currency of Impact

The most common fear engineers have about leaving the "Spotlight" is career stagnation. The logic goes: *If I'm not launching flashy features at Google I/O, and my work isn't on my VP's top 5 list, how will I ever get promoted to Staff+?*

It is true that you lose the currency of "Executive Visibility." But in infrastructure, you gain two alternate currencies that are just as valuable, and potentially more stable.

Shadow Hierarchy

In a product organization, you often need to impress your manager's manager. In an infrastructure organization, you need to impress your customers' managers.

I call this the Shadow Hierarchy. You don't need *your* VP to understand the intricacies of your code. You need the Staff+ Engineers in *other* critical organizations to need your tools.

When a Senior Staff Engineer in Pixel tells their VP, *"We literally cannot debug the next Pixel phone without Perfetto"*, that statement carries immense weight. It travels up their reporting chain, crosses over at the Director/VP level, and comes back down to your manager.

This kind of advocacy is powerful because it is technical, not political. It is hard to fake. When you are a steward of a critical system, your promotion packet is filled with testimonials from the most respected engineers in the company saying, *"This person's work enabled our success".*

Utility Ledger

While product teams might be poring over daily active users or revenue, we rely on metrics tracking engineering health:

- Utility: Every bug fixed using our tools is an engineer finding us useful. It is the purest measure of utility.

- Criticality: If the Pixel team uses Perfetto to debug a launch-blocking stutter, or Chrome uses it to fix a memory leak, our impact is implicitly tied to their success.
- Ubiquity: Capturing a significant percentage of the engineering population proves you've created a technical "lingua franca". This becomes especially obvious when you see disconnected parts of the company collaborating with each other, using shared Perfetto traces as a "reference everyone understands".
- Scale: Ingesting petabytes of data or processing billions of traces proves architectural resilience better than any design doc.

When you combine Criticality (VIP teams need this) with Utility (bugs are being fixed), you create a promotion case that is immune to executive reorganizations.

Archetypes and Agency

Staff Archetypes

I am far from the first to notice the idea of "there are multiple ways to be a staff software engineer". In his book *Staff Engineer*, Will Larson categorizes Staff-plus engineers into four distinct archetypes.

Sean describes the Solver or the Right Hand: engineers who act as agents of executive will, dropping into fires and moving on once the problem is stabilized. I am describing the Architect or the Tech Lead: roles defined by long-term ownership of a specific domain and deep technical context.

The "Luck" Rebuttal

I can hear the criticism already: *"You just got lucky finding your team. Most of us don't have that luxury."*

There are two caveats to all my advice in this post. First, the strategy I have employed so far requires a company profitable enough to sustain long-term infrastructure. This path generally does not exist in startups or early growth companies; it is optimized for Big Tech.

Second, luck *does* play a role in landing on a good team. It is very hard to accurately evaluate team and company culture from the outside. But while finding the team might have involved luck, staying there for almost a decade was a choice.

And, at least in my experience, my team is not particularly special: I can name five other teams in Android alone [4]. Sure, they might have a director change here or a VP change there, but the core mission and the engineering team remained stable.

The reason these teams seem rare is not that they don't exist, but that they are often ignored. Because they don't offer the rapid, visible "wins" of a product launch nor are they working on the "shiny cool features", they attract less competition. If you are motivated by "shipping to billions of users" or seeing your friends and family use something you built, you won't find that satisfaction here. That is the price of admission.

But if you want to build long-term systems and are willing to trade external validation for deep technical ownership, you just need to look behind the curtain.

Conclusion

The tech industry loves to tell you to move fast. But there is another path. It is a path where leverage comes from depth, patience, and the quiet satisfaction of building the foundation that others stand on.

You don't have to chase the spotlight to have a meaningful, high-impact career at a big company. Sometimes, the most ambitious thing you can do is stay put, dig in, and build something that lasts. To sit with a problem space for years until you understand it well enough to build a Bigtrace.

---

1. By product team I *don't* mean "frontend team": even as a backend engineer, you are still working on some part of what is being served directly to end users. ↩
2. This is not exhaustive, Perfetto is open source and we *do* also care about external developers but that's *not* why we get paid. From the company perspective, time we spent on open source bugs is "wasted" time but we do it because we believe in the mission of open source. I talked about this more in a recent post, On Perfetto, Open Source, and Company Priorities. ↩
3. For what it's worth, LLMs might not even be the best solution to "let's put AI into Perfetto": in my opinion there is lots of value with "old school" machine learning techniques like neural networks. A lot of trace analysis is just pattern matching. This is something I'm hoping to explore more in the coming year! ↩
4. Android Kernel, Android System Health, Android Runtime, Android Camera HAL, Android Bionic ↩

00:15 / #career #big-tech #software-engineering

We stopped roadmap work for a week and fixed 189 bugs →